

研发缺陷闭环协同系统 — 作品简介

项目名称

研发缺陷闭环协同系统 (RD Defect Closed-Loop Collaborative System)

问题与场景

当前 AI 辅助编码极大地提升了新功能开发速度，但缺陷修复仍高度依赖人工排查、定位与验证，形成“生成快、修复慢”的效率断层——团队快速堆积 AI 生成的代码，一旦出现缺陷，追溯根因、协同处理、回归验证完全跟不上生成速度，技术债务快速累积。此外，现代软件开发中缺陷修复流程还面临三大痛点：**分析慢**——开发者需要在海量代码与文档中追溯根因，耗时数小时；**修复易遗漏**——局部改动常引发回归，缺乏系统性验证；**协作碎片化**——分析、编码、测试、评审分散在多个工具中，信息传递损耗严重。尤其在大型项目维护场景，Issue 积压、Patch 质量参差、人工 Review 成为瓶颈。

核心解决方案

本系统基于 AgentTeams 多智能体协作框架构建了一条自动化的缺陷闭环流水线：**Manager**（任务调度）→ **Analyzer**（根因分析，基于 pgvector / Neo4j / Meiliseach 三库融合的语义检索）→ **Fixer**（代码修复，在隔离环境中生成 Patch）→ **Tester**（真实测试执行，在隔离 venv 中运行回归 + 针对性测试）→ **Evaluator**（质量门禁，产出结构化 verdict 报告）。系统通过 Matrix 协议实现 Agent 间消息派单，所有中间产物由共享存储流转，最终经 SWE-Bench 标准测试集进行全链路验证。

创新点与差异化优势

- 角色级模型分派**：Manager/Analyzer 使用 DeepSeek-V4-Pro (262K 长上下文)，Fixer/Tester 使用 Kimi-K2.7-Code (代码专用)，Evaluator 使用 DeepSeek-V3.2 (推理质量门禁)，按职责匹配模型能力与成本。
- 三库融合检索**：pgvector 向量相似度 + Neo4j 调用链路 + Meiliseach 模糊匹配，一工具暴露，语义检索召回率显著优于单库方案。

3. **真实测试执行**：Tester 非静态代码审查，而是动态检测语言→搭建隔离环境→安装兼容依赖→运行真实 `pytest/npm test`，将具体失败信息反馈 Fixer 形成迭代闭环。
4. **增量索引**：Redis 追踪 `file-hash`，仅索引变更文件，嵌入缓存复用，大幅降低重复计算成本。
5. **协议化协作**：Agent 间通过 Matrix 联邦协议通信，IM 通道天然支持多团队、多项目、人机协同扩展。
6. **全链路验证与灰度闭环**：SWE-Bench 标准测试集在隔离环境中验证修复正确性（F2P 必须全过 + P2P 不能打破）；设计预留了灰度发布结果确认机制——修补完成后生成 `release_plan.json`，部署到 canary 环境观察指标，根据 `confirmation_report.json` 自动判定发布或回滚，实现从“修复提交”到“安全上线”的完整闭环。
7. **经验沉淀与自进化闭环**：每次修复（无论成功或失败）的根因、修复策略、代码模式、踩过的坑被结构化写入 `lessons` 经验库（基于 `pgvector` 向量检索）。Analyzer 在根因分析时查询相似历史经验——相似度 ≥ 0.85 直接复用策略，0.60~0.85 作为参考注入 prompt；Fixer 在生成 patch 前查询历史改法以规避已知陷阱。写入前自动去重（MERGE/SIMILAR/NEW 三级决策），经验被多次独立验证后可信度权重递增。形成“修一次、学一次、越修越快”的自进化循环，经验可在任意产物（`fix.diff` / `verdict.json`）之间双向追溯。

开放/复用价值

系统采用严格的分层架构，各层职责清晰、边界明确：

- **声明式 Worker**：五角色通过 YAML 声明式定义（`soul + agents + skills + model`），新增语言或工具只需追加配置无需改代码。
- **Skill 包独立版本化**：15 个 AgentTeams Skills 打包为标准 ZIP（SemVer 版本控制），Worker YAML 通过 `spec.package` pinned 版本引用，支持独立升级、回滚和灰度发布（不同 Worker 可引用不同版本）。
- **MCP 协议层解耦**：`code-intelligence` MCP Server 暴露 13 个数据访问原语（`pgvector` 向量检索 / Neo4j 图谱遍历 / Meilisearch 关键词召回等）+ 1 个共享服务（`hybrid_search` RRF 融合），Skill 通过标准 MCP 协议调用底层数据能力，接口契约保持不变即可替换底层实现。
- **分层复用**：Skills 定义“怎么做”（业务编排），MCP 定义“能做什么”（数据能力），Worker 原生工具（`bash` / `git` / `file r/w`）由 runtime 直接提供——任意一层均可独立替换或跨项目复用，不相互耦合。
- **开源生态底座**：底层组件全部基于开源技术栈：`PostgreSQL + pgvector` / `Neo4j` / `Meilisearch` / `Redis` / `Matrix`，零商业许可依赖。

当前进展

已实现：完成 SWE-Bench 首个 Issue (pallets__flask-4045) 全链路验证——从增量索引到 5 Agent 协作修复，再到隔离 venv 中 F2P 2/2 + P2P 50/50 全部通过 (resolved)。系统核心流水线稳定运行，支持单题与批量自动化测试。

已完成详细设计，下一阶段实现：

- **全链路可观测性 (AgentLoop)**：设计了基于阿里云 AgentLoop 的三层监控方案——AgentTeams 平台自动采集 Worker 推理轨迹与 Skill 调用 Span；MCP Server 通过 OpenTelemetry 手动埋点，为每次数据访问原语生成子 Span；一次缺陷修复任务形成完整 Trace (task_id 贯穿 Worker → Skill → MCP 原语全部调用)，支撑链路追踪、异常审计与性能瓶颈分析。当前设计已落地文档，待 AgentTeams 平台接入后投入实现。
- **经验沉淀闭环 (Lessons Learned)**：设计了基于 pgvector 的 lessons 经验库——Evaluator 裁定后自动提取根因、修复策略、代码模式与踩坑记录写入经验库并向量化存储；写入前按相似度自动去重 (MERGE / SIMILAR / NEW 三级决策)；后续 Analyzer 与 Fixer 通过 lesson-lookup Skill 按相似度分级复用历史经验。形成“修一次、学一次、越修越快”的自进化闭环。
- **灰度发布与结果确认**：设计了事件驱动的灰度闭环流程——Manager 生成 release_plan.json 并创建 PR 后进入 awaiting_release 休眠态；由外部 CI/CD 完成构建与流量切分后，通过 webhook 事件唤醒 Manager，读取 confirmation_report.json 自动决策关单 (canary OK) 或回退 Analyzer 重新分析 (canary FAIL)；辅以超时哨兵机制防止静默失败。全链路遵循“Agent 修代码→人工审批发布→自动确认闭环”的协作边界。

上述三项设计的详细方案分别在 §4 (灰度发布)、§5 (经验沉淀)、§6 (全链路监控) 中展开，后续版本的开发将围绕这三条主线推进。